

Handleiding examen JavaScript — stappenplan

Een herbruikbaar stappenplan voor de examens van dit type (zie de oefenexamens 2025-06 "Quiz builder" en 2025-08 "Webshop"). Beide examens stellen telkens **dezelfde soort vragen**, in deze volgorde:

1. Routing + navbar
2. Eén item renderen (custom element + pagina haalt data op via de API)
3. Filteren
4. Verwijderen
5. Toevoegen aan een collectie (custom event) + opslaan in localStorage
6. Tweede pagina renderen (uit localStorage)
7. Updaten / verwijderen uit die collectie

Werk elke vraag apart af. Eerst renderen laten werken, dan pas de filter erbij, dan pas verwijderen, enz. Test na elke stap in de browser. Zo verlies je nooit punten van een werkende vraag door een latere fout.

Hoe deze handleiding is opgebouwd

Elke stap (en elk extra hoofdstuk) volgt **dezelfde vaste structuur**, zodat je snel je weg vindt:

1. **Doel** — wat de vraag oplevert (en hoeveel punten).
2. **Waar** — in welke bestanden je werkt.
3. **Hoe** — genummerde sub-stappen met kant-en-klare code (met  op de plekken die je zelf invult).
4. **De kern** — de paar dingen die je echt moet onthouden.
5.  **Let op** — de valkuilen die net daar misgaan.
6.  **Test** — hoe je controleert dat de stap werkt.

Je hoeft dit document niet van begin tot eind te lezen. Heb je enkel stap 5 nodig, spring er dan naartoe — elke stap is **zelfstandig leesbaar**. Daarom herhalen sommige waarschuwingen zich (bv. "deze naam moet exact gelijk zijn als daar"): die staan **op elke plek waar het misgaat**, met een verwijzing naar de andere plek, zodat je nooit afhangt van een stap die je hebt overgeslagen.

0. Plaatshouders die je overal invult

In dit document gebruik ik vaste namen. **Vervang ze door de namen van jouw examen:**

Plaatshouder	Betekenis	Voorbeeld 2025-06	Voorbeeld 2025-08
<code>Item</code>	de model-interface (1 ding uit de API)	<code>Question</code>	<code>Product</code>
<code>item / items</code>	variabele(n)	<code>question</code>	<code>product</code>
<code>custom-item</code>	tag van het item-component	<code>custom-question</code>	<code>custom-product-card</code>
<code>ItemCard</code>	klasse van het item-component	<code>QuestionComponent</code>	<code>CustomProductCard</code>
<code>ItemsPage</code>	klasse van de hoofdpagina	<code>HomePage</code>	<code>ProductPage</code>
<code>itemRestProvider</code>	REST-provider (API)	<code>questionPersistenceProvider</code>	<code>productRestPersistenceProvider</code>
<code>API_URL</code>	de API-route	<code>http://localhost:3000/questions</code>	<code>http://localhost:3000/products</code>
<code>collectionLocalProvider</code>	localStorage-provider	<code>quizPersistenceProvider</code>	<code>cartLocalPersistenceProvider</code>
<code>STORAGE_KEY</code>	de localStorage-sleutel	<code>'quizzes'</code>	<code>'cart'</code>

Belangrijkste regel om te onthouden (komt overal terug): Een pagina doet 3 dingen → **(a)** observer registreren, **(b)** `getAll()` oproepen, **(c)** in `render()` per item een custom element maken en de data via **attributen** (strings, kebab-case!) doorgeven.

Het framework dat je KRIJGT (niet aanpassen, wel gebruiken)

- `CustomElement` — basisklasse voor een herbruikbaar element. Jij maakt een subklasse met `observedAttributes` (lijst van attributen die je in de gaten houdt) + `attributeChangedCallback` (reageert op wijzigingen) + eventueel `dispatchEvent(new CustomEvent(...))` voor custom events.
 - `Page` — basisklasse voor een pagina. `render()` tekent de pagina; `unsubscribe` ruimt observers op.
 - `Router` — koppelt URL-paden aan pagina's; leest bij opstart `window.location.pathname`.
 - `PersistenceProvider` — databron met **observer-patroon**: `addObserver(cb)` geeft je een callback die loopt telkens de data wijzigt. Twee soorten:
 - `RestPersistenceProvider` → praat met de API (fetch). Methodes: `getAll`, `get`, `create`, `update`, `delete`.
 - `LocalStoragePersistenceProvider` → bewaart in `localStorage` onder een `key`. Zelfde methodes.
-

STAP 1 — Routing + navbar (meestal 1 punt)

Doel: custom elements registreren, navbar bovenaan elke pagina, links werken, juiste paden.

Waar: `main.ts` (registreren + routes), elke `pages/<naam>/<naam>.ts` + `.html`, en `components/navbar/navbar.ts` + `.html`.

1a. EERST: maak een minimaal TS-bestand per pagina en per custom element

Belangrijk: in de startbestanden krijg je wél de **HTML** van de pagina's en de custom elements (`*.html`), maar **niet** altijd het bijhorende **.ts -bestand**. Die moet je dus zelf aanmaken — anders kan `main.ts` ze niet importeren, kan de router de pagina niet aanmaken en zie je in vraag 1 **niets**. De opgave-tip zegt dit ook: voorzie voor elke pagina/component eerst een TS-bestand dat **enkel de HTML toont**; de echte inhoud werk je later af.

Minimaal pagina-bestand (bv. `pages/items/items.ts`). De basisklasse `Page` toont via `super(HTML)` + de standaard `render()` al de HTML — meer heb je voor vraag 1 niet nodig:

```
import {Page} from '../../router/page.ts'
import HTML from './items.html?raw'

export class ItemsPage extends Page {
  constructor() {
    super(HTML)
  }
  // render() vul je later aan (stap 2). Voorlopig volstaat de render() van de basisklasse.
}
```

Doe hetzelfde voor de tweede pagina (`pages/second/second.ts` → `SecondPage`).

Minimaal custom-element-bestand (bv. `components/itemCard/itemCard.ts`):

```
import {CustomElement} from '../../router/customElement.ts'
import HTML from './itemCard.html?raw'

export class ItemCard extends CustomElement {
  constructor() {
    super(HTML)
  }
  // observedAttributes + attributeChangedCallback komen er later bij (stap 2).
}
```

Doe dit voor élk custom element dat je in `main.ts` wil registreren (navbar, item-kaart, ...).

Zo is alles importeerbaar en kan je meteen testen. In de volgende sub-stappen vul je `main.ts` aan; vanaf stap 2 breid je deze stubs uit met de echte inhoud.

1b. `src/main.ts` — registreren + router

```
import 'bootstrap'
import 'bootstrap/dist/css/bootstrap.css'
import {Router} from './router/router.ts'

// 🐞 importeer je pagina's en custom elements
import {ItemsPage} from './pages/items/items.ts'
import {SecondPage} from './pages/second/second.ts'
import {CustomNavbar} from './components/navbar/navbar.ts'
import {ItemCard} from './components/itemCard/itemCard.ts'

// 🐞 registreer ELK custom element (de tag-naam mag je vaak zelf kiezen, navbar soms verplicht)
window.customElements.define('custom-navbar', CustomNavbar)
window.customElements.define('custom-item', ItemCard)

// 🐞 koppel elk pad aan de juiste pagina
new Router({
  '/': ItemsPage,
  '/second': SecondPage,
})
```

🔗 **Onthoud de tag-namen die je hier kiest** (bv. `'custom-item'`). Je gebruikt ze **letterlijk opnieuw** wanneer je in een pagina een element aanmaakt met `document.createElement('custom-item')` (stap 2c en stap 6). Die strings **moeten exact gelijk** zijn — een andere naam of typefout geeft een leeg, onbekend element (je ziet niets, zonder foutmelding).

1c. `components/navbar/navbar.ts` — minimaal element (als het nog niet bestaat)

```
import {CustomElement} from '../../router/customElement.ts'
import HTML from './navbar.html?raw'

export class CustomNavbar extends CustomElement {
  constructor() {
    super(HTML)
  }
}
```

1d. De navbar bovenaan elke pagina-HTML zetten

In `pages/items/items.html` en `pages/second/second.html`, bovenaan:

```
<custom-navbar></custom-navbar>
```

1e. De links in `navbar.html` laten werken

Twee mogelijkheden (kies wat in de startcode staat):

- **Met router (zonder herladen):** `data-link` toevoegen → de Router maakt er een werkende link van.

```
<a href="/" data-link="/">Eerste</a>
<a href="/second" data-link="/second">Tweede</a>
```

- **Gewone links (volledige herlaad):** gewoon het juiste pad in `href` zetten.

```
<a href="/">Eerste</a>  
<a href="/second">Tweede</a>
```

✓ **Test:** beide pagina's openen via de navbar.

STAP 2 — Eén item renderen (vaak de zwaarste vraag: 4-5 punten)

Doel: alle items via de API ophalen (verplicht via `RestPersistenceProvider`) en tonen met een zelfgebouwd custom element per item.

Waar: `data/data.ts` (provider), `components/itemCard/itemCard.ts` (het component), `pages/<eerste-pagina>.ts` (de pagina).

2a. Provider aanmaken in `src/data/data.ts`

```
import {RestPersistenceProvider} from './restPersistenceProvider.ts'
import type {Item} from '../models/item.ts'

// 🐞 API_URL = de route uit de opgave
export const itemRestProvider = new RestPersistenceProvider<Item>('API_URL')
```

⚠️ **Het type tussen `<` `>` is verplicht en niet optioneel!** Vergeet je het (`new RestPersistenceProvider('...')` of `new LocalStoragePersistenceProvider('...')`), dan weet TypeScript niet welke objecten de provider bevat en valt het terug op een leeg type. Gevolg: bij `create({ name: ... })` krijg je **"property 'name' bestaat niet"** — je velden worden niet herkend. Zet er dus altijd je model-type in: `<Item>`, `<Quiz>`, `<CartItem>`, ...

► **Kies hier een MODEL uit `models/`, niet een component-klasse uit `components/`.** Een provider bewaart **data**, geen UI-elementen. Let op met autocomplete: voor het winkelmandje wil je `<CartItem>` (het model), **niet** `<CustomProduct>` / `<ProductCard>` (je custom element). Importeer je per ongeluk de component-klasse, dan krijg je later een fout als *"Type 'CustomProduct[]' is not assignable to type 'CartItem[]' — Property 'product' is missing"*, want je observer geeft dan het verkeerde type terug.

2b. Het custom element `components/itemCard/itemCard.ts`

Tip uit de opgave: je kan enkel **strings** doorgeven aan een custom element, en de attribuut-namen moeten in **kebab-case** staan (`correct-answer`, niet `correctAnswer`).

Welke attributen zet ik in `observedAttributes`? Twee soorten:

1. **De datavelden die je wil tonen.** Kijk naar de JSON die de API teruggeeft (ga met je browser naar de `API_URL`, bv. `http://localhost:3000/movies`, of bekijk de `Item`-interface in `models/`). Neem daarvan de velden die je in de HTML zichtbaar wil maken — meestal alles behalve `id`. Voorbeeld voor een film: `title`, `genre`, `year`, `rating`, `director`. **Schrijf ze in kebab-case** (een JSON-veld `correctAnswer` wordt het attribuut `correct-answer`).
2. **Extra "status"-attributen die je zelf verzint** en die níét in de JSON staan, maar die je nodig hebt om de UI te sturen. Voorbeelden: `is-added` / `in-watchlist` (zit het item al in de collectie? → knop toont + of ✓), of `hide-delete` (vuilbak verbergen op de tweede pagina). Die voeg je toe naargelang de opgave erom vraagt.

Regel: elk attribuut dat je in de pagina met `setAttribute(...)` zet (zie stap 2c), moet hier in `observedAttributes` staan **én** een `case` krijgen in `attributeChangedCallback` — anders gebeurt er niets. (`id` is een uitzondering: dat zet je wel met `setAttribute('id', ...)`, maar het hoeft niet in `observedAttributes` omdat je het enkel gebruikt om te verwijderen/updaten, niet om iets te tonen.)

```

import HTML from './itemCard.html?raw'
import {CustomElement} from '../../router/customElement.ts'

export class ItemCard extends CustomElement {
  // 🐞 alle attributen die je wil tonen/observeren (kebab-case!)
  static observedAttributes = ['name', 'price', 'category']

  // 🐞 verwijzingen naar de plekken in itemCard.html (id's uit de HTML)
  #name = this.componentBody.querySelector<HTMLElement>('#name')!
  #price = this.componentBody.querySelector<HTMLElement>('#price')!
  #category = this.componentBody.querySelector<HTMLElement>('#category')!

  constructor() {
    super(HTML)
  }

  attributeChangedCallback(name: string, _oldValue: string, newValue: string) {
    switch (name) {
      case 'name':
        this.#name.innerText = newValue
        break
      case 'price':
        // 🐞 getal mooi tonen (newValue is een string!)
        this.#price.innerText = Number(newValue).toFixed(2) + ' EUR'
        break
      case 'category':
        this.#category.innerText = newValue
        break
      // 🐞 ARRAY tonen? Geef hem als JSON-string door en parse hem hier:
      // case 'answers': {
      //   this.#list.innerHTML = ''
      //   JSON.parse(newValue).forEach((x: string) => {
      //     const li = document.createElement('li'); li.innerText = x; this.#list.appendChild(li)
      //   })
      //   break
      // }
    }
  }
}

```


Welk `<HTML ... Element>` -type bij welke tag? Bij `querySelector< ... >('#id')` zet je tussen `< >` het type dat hoort bij het HTML-element in je `.html`. Kijk in je HTML welk element dat `id` heeft en kies het juiste type:

HTML-element	Waarvoor	TypeScript-type
<code><h1></code> – <code><h6></code>	titel / kop	<code>HTMLHeadingElement</code>
<code></code>	klein stukje tekst inline	<code>HTMLSpanElement</code>
<code><p></code>	paragraaf (tekstblok)	<code>HTMLParagraphElement</code>
<code><div></code>	algemene container	<code>HTMLDivElement</code>
<code></code> / <code></code>	lijst (waarin je <code></code> 's steekt)	<code>HTMLUListElement</code> / <code>HTMLOListElement</code>
<code></code>	één lijst-item	<code>HTMLLIElement</code>
<code><button></code>	knop	<code>HTMLButtonElement</code>
<code><input></code>	tekstveld / getal / radio / checkbox	<code>HTMLInputElement</code>
<code><select></code>	dropdown	<code>HTMLSelectElement</code>
<code><form></code>	formulier	<code>HTMLFormElement</code>
<code><a></code>	link	<code>HTMLAnchorElement</code>
<code></code>	afbeelding	<code>HTMLImageElement</code>
<code><label></code>	label bij een input	<code>HTMLLabelElement</code>

Twijfel je of weet je het niet? `HTMLElement` werkt altijd (dat is de gemeenschappelijke basis). Je verliest dan enkel type-specifieke extra's: `.value` (op `input` / `select`), `.disabled` / `.hidden` (op `button`), `.checked` (op een checkbox-`input`). Heb je die nodig, kies dan het specifieke type. Voor enkel `.innerText` / `.innerHTML` volstaat `HTMLElement` of gewoon `HTMLSpanElement`.

2c. De pagina `pages/items/items.ts`

```
import {Page} from '../router/page.ts'
import HTML from './items.html?raw'
import type {Item} from '../models/item.ts'
import {itemRestProvider} from '../data/data.ts'
import {ItemCard} from '../components/itemCard/itemCard.ts'

export class ItemsPage extends Page {

  #items: Item[] = []
  // 🐞 de container uit items.html waarin de items komen
  #container = this.body.querySelector<HTMLDivElement>('#items')!

  constructor() {
    super(HTML)

    // (a) observer: bij elke datawijziging de nieuwe lijst opslaan + herrenderen
    this.unsubscribe.push(itemRestProvider.addObserver(items => {
      this.#items = items
      this.render()
    }))

    // (b) ophalen via de API → verwittigt de observer → UI vult zich
    void itemRestProvider.getAll()
  }

  render(): void {
    super.render()

    // (c) container leegmaken en per item een custom element bouwen
    this.#container.innerHTML = ''
    this.#items.map(item => {
      const el = document.createElement('custom-item') // 🐞 ZELFDE naam als bij customElements.define in main.ts
      el.setAttribute('id', item.id) // id altijd meegeven (nodig voor verwijderen!)
      el.setAttribute('name', item.name)
      el.setAttribute('price', item.price.toFixed(2))
      el.setAttribute('category', item.category)
      // 🐞 array? → el.setAttribute('answers', JSON.stringify(item.answers))

      this.#container.appendChild(el)
    })
  }
}
```

Belangrijk — de naam in `createElement('custom-item')` moet exact dezelfde tag-naam zijn als die je in `main.ts` registreerde met `customElements.define('custom-item', ItemCard)` (zie stap 1b). Schrijf je hier een andere naam (of een typefout), dan maakt de browser een leeg, onbekend element aan: je ziet niets en krijgt geen duidelijke foutmelding.

```
// main.ts
window.customElements.define('custom-item', ItemCard) // ← deze string ...
// items.ts
const el = document.createElement('custom-item') // ← ... moet hier exact gelijk zijn
```

Alternatief: `const el = new ItemCard()` (de klasse rechtstreeks). Dat werkt ook en is typeveiliger, maar dan moet de klasse wél geïmporteerd én geregistreerd zijn.

 Elke attribuutnaam die je hier met `setAttribute('naam', ...)` zet, moet exact gelijk zijn aan een naam in `observedAttributes` (+ een `case 'naam':`) in je component (stap 2b). Komt een naam niet overeen (of staat hij niet in `observedAttributes`), dan loopt `attributeChangedCallback` er niet voor en blijft dat veld leeg. Schrijf alles in **kebab-case** (`correct-answer`, niet `correctAnswer`), want de browser maakt attribuutnamen sowieso lowercase.

✓ **Test:** alle items verschijnen op de pagina.

STAP 3 — Filteren (2-3 punten)

Doel: filteren op tekst (input) en/of categorie (dropdown) en/of type (radio). Combinaties moeten samen werken.

Waar: enkel op de **eerste pagina** (`pages/<eerste-pagina>.ts` + de bijbehorende `.html` met de filtervelden).

Let op de eis: soms moet je filteren **bij elke wijziging** (`change` -event op input/select/radio), soms pas **op een knop** (`click` op de filterknop). Lees de opgave! En bij tekst: zoeken op een **deel** van de naam en **niet** hoofdlettergevoelig = `.toLowerCase().includes(...)`.

3a. Filterstatus + listeners bijhouden in `ItemsPage`

```
// ↗ verwijzingen naar de filter-elementen
#nameFilter = this.body.querySelector<HTMLInputElement>('#name-filter')!
#categoryFilter = this.body.querySelector<HTMLSelectElement>('#category-filter')!
#filterBtn = this.body.querySelector<HTMLButtonElement>('#filter-btn')!
```

In de constructor, kies één van beide. Zet de listener **rechtstreeks in de constructor**, naast je observers — **niet** binnen een observer-callback (anders voeg je telkens opnieuw een listener toe elke keer de data binnenkomt):

```
constructor() {
  super(HTML)

  // ... hier je observers + getAll() (stap 2) ...

  // Variant A: filteren op een KNOP (form → preventDefault tegen herladen)
  this.#filterBtn.addEventListener('click', evt => {
    evt.preventDefault()
    this.render()
  })

  // Variant B: filteren bij ELKE wijziging
  // this.#nameFilter.addEventListener('change', () => this.render())
  // this.#categoryFilter.addEventListener('change', () => this.render())
}
```

3b. In `render()` filteren via een aparte hulpfunctie

```
render(): void {
  super.render()
  this.#container.innerHTML = ''
  this.#items
    .filter(item => this.#matchesFilter(item)) // ✎ enkel deze regel toevoegen t.o.v. stap 2
    .map(item => { /* ... zelfde als stap 2 ... */ })
}

// ✎ aparte functie houdt het leesbaar (tip uit de opgave)
#matchesFilter(item: Item): boolean {
  const nameMatch = item.name.toLowerCase().includes(this.#nameFilter.value.toLowerCase())
  // categorie '0' of '' betekent meestal "alles" → dan negeren we de categoriefilter
  const categoryMatch =
    this.#categoryFilter.value === '0' ||
    item.category.toLowerCase() === this.#categoryFilter.value.toLowerCase()

  return nameMatch && categoryMatch // alle filters via && => werken samen
}
```

Radio-knoppen (type-filter, zoals 2025-06): ⚠ Een radio- `input` z'n `.value` is **altijd dezelfde** (de vaste `value`-attribuutwaarde), ook als hij niet aangevinkt is. `querySelector('#true-false').value` geeft je dus nooit de keuze van de gebruiker. Lees in plaats daarvan de **aangevinkte** radio met `:checked`. Radio's die bij elkaar horen delen dezelfde `name` (bv. `name="questionTypes"`):

```
// verwijzing naar ALLE radio's van die groep
#typeFilters = this.body.querySelectorAll<HTMLInputElement>('input[name="questionTypes"]')

// in de constructor: naar elke radio luisteren
this.#typeFilters.forEach(radio => radio.addEventListener('change', () => this.render()))

// in #matchesFilter: lees de radio die NU aangevinkt is
const selectedType = this.body.querySelector<HTMLInputElement>('input[name="questionTypes"]:checked')!.value
const typeMatch = item.type === selectedType
```

(Alternatief: houd een eigen veld `#selectedType` bij en zet dat in een `change`-listener per radio.) Bij een **dropdown** (`<select>`) werkt `.value` wél meteen — die valkuil geldt enkel voor radio's/checkboxes.

✓ **Test:** filter op naam, op categorie, en op de combinatie.

STAP 4 — Verwijderen uit de database (2 punten)

Doel: op een knop (vuilbakje) het item via de API verwijderen, en de UI bijwerken. **Verplicht via** `RestPersistenceProvider`.

Waar: in het **item-component** van de eerste pagina (`components/itemCard/itemCard.ts`) — de knop zit in het component.

In het custom element `itemCard.ts` (de knop zit in het component):

```
import {itemRestProvider} from '../..//data/data.ts' // bovenaan toevoegen

// verwijzing naar de knop
#deleteBtn = this.componentBody.querySelector<HTMLButtonElement>('#delete')!

constructor() {
  super(HTML)
  // this.id = het 'id'-attribuut dat de pagina meegaf
  this.#deleteBtn.addEventListener('click', () => itemRestProvider.delete(this.id))
}
```

Waarom werkt de UI vanzelf bij? `delete()` verwittigt de observers → de pagina herrendert automatisch. **Daarom moet je in stap 2 het `id` altijd als attribuut meegeven.**

✓ **Test:** klik op verwijderen → item verdwijnt en blijft weg na refresh.

STAP 5 — Toevoegen aan een collectie via CUSTOM EVENT + opslaan in localStorage (3-4 punten)

📌 **WAAR gebeurt dit? Op de EERSTE pagina** (dezelfde als stap 2-4: de lijst met item-kaarten, bv. de catalogus/producten/recepten-pagina) **en in de item-kaart van die pagina**. Hier voeg je items TOE aan de collectie. De tweede pagina (waar je de collectie *toont* en items eruit haalt) is **stap 6 en 7** — niet hier.

Concreet werk je in stap 5 dus in: `pages/<eerste-pagina>.ts` (sub-stappen 5a, 5c, 5d) en in `components/<item-kaart>.ts` (sub-stap 5b). **Niet** in de tweede-pagina-bestanden.

Doel: met een knop selecteer je een item / voeg je het toe aan een mandje of quiz. Gebruik een **custom event** (het component mag niet zelf aan de pagina-data zitten). Bewaren in **localStorage** via `LocalStoragePersistenceProvider`. De knop toont een andere staat als het item al toegevoegd is (+ ↔ - of ✓).

Waar: `components/itemCard/itemCard.ts` (5b) + `pages/<eerste-pagina>.ts` (5a, 5c, 5d) + `data/data.ts` (5a). Zie ook het 📌-kader hierboven.

Wat is een custom event en hoe werkt het? (2 helften) Een custom element (het kind) mag niet rechtstreeks aan de data van de pagina (de ouder) zitten. Daarom **roept** het component iets als er geklikt wordt, en de pagina **luistert** daarnaar:

1. **In het component (de kaart) — afvuren:** in de click-listener van de knop:

```
this.#addBtn.addEventListener('click', () => {
  this.dispatchEvent(new CustomEvent('addToCollection')) // naam kies je zelf
})
```

2. **In de pagina — luisteren:** op het element dat je per item aanmaakt:

```
el.addEventListener('addToCollection', () => { // EXACT dezelfde naam als bij dispatchEvent
  // hier pas je de data aan (Set of provider) en herrender je
})
```

Dat is alles: `dispatchEvent` in de kaart ↔ `addEventListener` met dezelfde naam in de pagina. Wát je in de listener doet, hangt af van de opgave → zie **aanpak A** (direct in localStorage) of **aanpak B** (een `Set`) hieronder bij 5c.

🌟 **Vraag "Vragen toevoegen aan een quiz" (2025-06)?** Dat is precies dit, met **aanpak B (Set)**: je verzamelt de geselecteerde id's in een `Set`, want de quiz wordt pas later echt opgeslagen bij "Quiz aanmaken" (5d). De knop toont - als de vraag al geselecteerd is, anders + (via een `selected`-attribuut, zie 5b).

5a. Extra localStorage-provider in `data.ts`

```
import {LocalStoragePersistenceProvider} from './localStoragePersistenceProvider.ts'
import {Collection} from '../models/collection.ts' // 🐞 bv. Quiz of CartItem

// 🐞 STORAGE_KEY uit de opgave (bv. 'quizzes' of 'cart')
export const collectionLocalProvider = new LocalStoragePersistenceProvider<Collection>('STORAGE_KEY')
```

5b. Knop vuurt een custom event af in `itemCard.ts`

```
static observedAttributes = [/* ... */ , 'is-added'] // 🐞 'is-added' toevoegen
#addBtn = this.componentBody.querySelector<HTMLButtonElement>('#add')!

constructor() {
  super(HTML)
  // 🐞 stuur een custom event naar de parent (de pagina luistert hierop)
  // De NAAM ('addToCollection') mag je zelf kiezen, MAAR je moet exact dezelfde naam gebruiken
  // wanneer de pagina erop luistert (zie 5c). Kies dus nu een naam en onthoud die.
  this.#addBtn.addEventListener('click', () => {
    this.dispatchEvent(new CustomEvent('addToCollection'))
  })
}
```

De knopstaat tonen in `attributeChangedCallback`. De pagina zet `is-added` op `'true'` of `'false'` (zie 5c). In het component vang je dat op en pas je het uiterlijk van de knop aan. Voeg dus een `case 'is-added'` toe aan je bestaande switch:

```
attributeChangedCallback(name: string, _oldValue: string, newValue: string) {
  switch (name) {
    // ... je andere cases (title, genre, ...) ...

    case 'is-added':
      if (newValue === 'true') {
        // item zit al in de collectie → groene knop met een vinkje
        this.#addBtn.setAttribute('class', 'btn btn-success w-100')
        this.#addBtn.innerHTML = '&check; Toegevoegd' // 🐞 jouw tekst
      } else {
        // nog niet → gewone knop met een +
        this.#addBtn.setAttribute('class', 'btn btn-primary w-100')
        this.#addBtn.innerHTML = '+ Toevoegen' // 🐞 jouw tekst
      }
      break
  }
}
```

Belangrijke details:

- **Wil je een HTML-symbool tonen** zoals `✓` (✓), gebruik dan `innerHTML`, NIET `innerText`. Met `innerText` verschijnt letterlijk de tekst `✓` i.p.v. een vinkje.
- **Alleen de tekst wisselen mag ook** (eenvoudiger), als de opgave geen kleurwijziging vraagt:
`this.#addBtn.innerHTML = newValue === 'true' ? '✓' : '+'`
- De **- / + -variant** (zoals bij een quiz, waar je toevoegt én verwijdert via dezelfde knop): `this.#addBtn.innerText = newValue === 'true' ? '-' : '+'`
- `setAttribute('class', '...')` **overschrijft** alle classes van de knop. Zet er dus de volledige set Bootstrap-classes in (bv. `btn btn-success w-100`), niet enkel de kleur.
- Vergeet niet `'is-added'` in `observedAttributes` te zetten (zie hierboven), anders draait deze `case` nooit.

5c. De pagina luistert op het event + zet de knopstaat

Er zijn **twee manieren**. Lees de opgave: moet het meteen **bewaard blijven** (ook na refresh), dan is het **AANPAK A (localStorage)**. Verzamel je items om er later in één keer iets van te maken (bv. een quiz "aanmaken" met een knop), dan is het **AANPAK B (Set in het geheugen)**.

AANPAK A — direct in localStorage bewaren (meest gevraagd: winkelmandje, kijklijst, weekmenu)

Hier is **de localStorage-lijst zélf je waarheid** — je houdt geen aparte **Set** bij. Je hebt drie dingen nodig. Dit komt allemaal in de klasse van je **EERSTE pagina** (bv. `movies.ts` / `products.ts`), náást de observer op je API-data uit stap 2.

💡 **Verwar dit niet met stap 6.** Op de eerste pagina (hier) zet je een observer op de localStorage-provider zodat de **+/-knop** klopt. Op de tweede pagina (stap 6) zet je óók zo'n observer, maar daar om de **lijst te tonen**. Twee verschillende pagina's, allebei met een observer op dezelfde provider — dat is normaal.

(1) Een observer + `getAll()` op de localStorage-provider in de constructor (exact hetzelfde patroon als voor je API-data in stap 2; je hebt dit nodig zodat de knopstaat klopt en meteen herrendert na een wijziging):

```
#collection: Collection[] = [] // ↗ bv. WatchListItem[] / CartItem[]

// in de constructor:
this.unsubscribe.push(collectionLocalProvider.addObserver(data => {
  this.#collection = data
  this.render()
}))
void collectionLocalProvider.getAll()
```

(2) Per item: bepaal of het al in de collectie zit en zet het `is-added`-attribuut. Je zoekt met `find` of er al een collectie-item bestaat dat naar dit item verwijst:

```
// in render(), binnen je .map(item => { ... }):
const existing = this.#collection.find(c => c.product.id === item.id) // ↗ c.product/c.movie/... = jouw v
el.setAttribute('is-added', existing ? 'true' : 'false')
```

(3) Luister op het custom event en roep `create` of `delete` op de provider aan. Zit het er al in → `delete(existing.id)`, anders → een nieuw collectie-object `create(...)`:

```
// ⚠ De event-naam hier moet EXACT dezelfde zijn als die je in je kaart koos (5b)!
// Schrijf je hier een andere naam, dan gaat de listener nooit af en gebeurt er niets bij het klikken.
el.addEventListener('addToCollection', async () => { // ← zelfde string als in 5b (dispatchEvent)
  if (existing) {
    await collectionLocalProvider.delete(existing.id) // zit er al in → eruit halen
  } else {
    // ↗ bouw je collectie-object. Het 'id' is dat van de COLLECTIE-regel (niet van het item zelf).
    const newEntry: Collection = { product: item, id: crypto.randomUUID() }
    await collectionLocalProvider.create(newEntry) // nog niet → toevoegen
  }
})
```

Veelgemaakte fout (= "er gebeurt niets bij klikken"): in je kaart `dispatchEvent(new CustomEvent('X'))` maar op de pagina `addEventListener('Y', ...)` met een andere naam. `'X'` en `'Y'` moeten **identiek** zijn. Tip: kopieer de naam letterlijk van de ene plek naar de andere.

Hoe werkt dit precies?

- Je schrijft **nooit zelf** `localStorage.setItem`. De `LocalStoragePersistenceProvider` doet dat in zijn `create / delete / update`, en verwittigt daarna zijn observers.
- Door die observer (stap 1) draait `render()` opnieuw → de knop wisselt meteen naar ✓ en de tweede pagina (stap 6) toont meteen het nieuwe item. Geen `this.render()` nodig in de listener zelf.
- **existing.id vs item.id**: een collectie-item (bv. `CartItem / WatchListItem`) heeft een **eigen id** en bevat het item (`{ id, product }`). Je verwijdert met het id van de **collectie-regel** (`existing.id`), en je zoekt of het er al in zit via het id van het **item** (`c.product.id === item.id`).
- **Geen Set en geen #selectedIds** in deze aanpak — die horen bij aanpak B hieronder.

AANPAK B — een `Set` in het geheugen (alleen om later iets aan te maken, bv. een quiz)

Gebruik dit enkel als je items "verzamelt" en pas later met een aparte knop opslaat (zie 5d). Je bewaart de geselecteerde id's in een `Set` en schrijft (nog) niets naar `localStorage`:

```
#selectedIds: Set<string> = new Set() // bovenaan de klasse

// in render(), per item:
el.setAttribute('is-added', this.#selectedIds.has(item.id).toString())
el.addEventListener('addToCollection', () => {
  this.#selectedIds.has(item.id)
    ? this.#selectedIds.delete(item.id)
    : this.#selectedIds.add(item.id)
  this.render() // hier WEL zelf herrenderen (geen provider die observers verwittigt)
})
```

5d. "Aanmaken"-knop (bv. quiz aanmaken) — enkel bij dit type opgave

Dit gebruik je bij **aanpak B**: je verzamelde items in een `Set` en slaat ze nu in één keer op als één nieuw object (bv. een quiz met een naam + de gekozen vragen). Hieronder gebruikte velden — **maak ze eerst aan** bovenaan je klasse, met de echte id's uit jouw HTML:

```
// 🐞 velden die je hiervoor nodig hebt (bovenaan de klasse):
#selectedIds: Set<string> = new Set() // de geselecteerde id's (zie 5)
#nameInput = this.body.querySelector<HTMLInputElement>('#quiz-name')! // 🐞 het invoerveld voor de naam
#createBtn = this.body.querySelector<HTMLButtonElement>('#create-quiz')! // 🐞 de "Create quiz"-knop (bv.)
```

```
// in render(): knop disabled als er niets geselecteerd is
this.#createBtn.disabled = this.#selectedIds.size === 0
```

```
// in de constructor: bij klik opslaan in localStorage en daarna alles leegmaken
this.#createBtn.addEventListener('click', () => {
  void collectionLocalProvider.create({
    name: this.#nameInput.value, // de getypte naam uit het invoerveld
    items: this.#items.filter(i => this.#selectedIds.has(i.id)), // de geselecteerde items (jouw structu
  })
  this.#selectedIds = new Set() // selectie leegmaken
  this.#nameInput.value = '' // invoerveld leegmaken
  this.render()
})
```

`#nameInput` = dus gewoon een verwijzing naar het tekstveld waarin de gebruiker de naam typt (zoals `#nameFilter` / `#container` verwijzingen naar andere HTML-elementen zijn). `.value` leest wat erin staat, en `''` maakt het terug leeg. Gebruik het id dat in jóúw HTML staat (in de quiz-opgave: `#quiz-name`).

✓ **Test:** item toevoegen → knop wisselt; collectie aanmaken → opgeslagen in localStorage (zie DevTools → Application → Local Storage).

STAP 6 — Tweede pagina renderen uit localStorage (1-4 punten)

Doel: op de tweede pagina de opgeslagen collectie tonen. Soms hergebruik je hetzelfde item-component, soms is er een **apart** component voor deze pagina (bv. `CartItem` / `WatchListItem`).

Waar: `pages/<tweede-pagina>.ts` + `.html`, en het collectie-item-component (`components/cartItem/...` of `watchListItem/...`).

⚠ Twee veelgemaakte fouten hier:

1. **De id's in onderstaande code zijn PLAATSHOUDERS.** `'#container'`, `'custom-item'`, `entry.product ...` enz. moet je vervangen door de **echte id's en namen uit JOUW HTML/model**. Open je `second.html` en kijk welk id de lijst-container heeft (bv. `id="watchlist"` → `querySelector('#watchlist')`, NIET `'#container'`). Een verkeerd id geeft `null` → `render()` crasht → je ziet niets.
2. **Het item-component moet zelf zijn `attributeChangedCallback` hebben** (zie stap 2b). Als je voor deze pagina een apart component gebruikt dat nog een lege stub is, verschijnen je items wel maar **zonder inhoud** (de standaardtekst uit de HTML). Werk dat component af net zoals je item-kaart.

In `pages/second/second.ts` — exact hetzelfde patroon als stap 2, maar met `collectionLocalProvider`. Hier het **volledige bestand** (let op de imports bovenaan en de velden in de klasse):

```

import {Page} from '../router/page.ts'
import HTML from './second.html?raw'
// 🐞 importeer je localStorage-provider (uit data.ts) en het type van je collectie-item
import {collectionLocalProvider} from '../data/data.ts'
import type {Collection} from '../models/collection.ts' // 🐞 bv. CartItem / WatchlistItem

export class SecondPage extends Page {

  // 🐞 de array die de collectie uit localStorage bijhoudt - MOET je zelf aanmaken
  #collection: Collection[] = []
  // 🐞 de container uit second.html - VERVANG '#container' door het echte id uit JOUW html (bv. '#watchlist')
  #container = this.body.querySelector<HTMLUListElement>('#container')!

  constructor() {
    super(HTML)

    // observer: bij elke wijziging de nieuwe lijst opslaan + herrenderen
    this.unsubscribe.push(collectionLocalProvider.addObserver(data => {
      this.#collection = data
      this.render()
    }))

    // inladen uit localStorage → verwittigt de observer → render()
    void collectionLocalProvider.getAll()
  }

  render(): void {
    super.render()
    this.#container.innerHTML = ''

    // totaalprijs/aantal? → reduce (zie tip onder)
    // const total = this.#collection.map(x => x.product.price).reduce((a, b) => a + b, 0)

    this.#collection.map(entry => {
      const el = document.createElement('custom-item') // 🐞 ZELFDE naam als bij customElements.define in main.ts
      el.setAttribute('id', entry.id)
      // 🐞 naam (+ extra info) op één regel met een TEMPLATE LITERAL (komt vaak terug):
      el.setAttribute('title', `${entry.product.name} (${entry.product.price.toFixed(2)} EUR)`)
      this.#container.appendChild(el)
    })
  }
}

```

De naam in `createElement('custom-item')` moet exact de tag-naam zijn die je in `main.ts` registreerde met `customElements.define('custom-item', ...)` (zie stap 1b). Op deze tweede pagina is dat meestal het **aparte** component, bv. `customElements.define('custom-watchlist-item', ...)` → `document.createElement('custom-watchlist-item')`. Een andere naam of typefout = een leeg, onbekend element (je ziet niets).

Waar komt de regel `el.setAttribute('title', \...)` vandaan? En wat is 'title'? Die regel is de ontmoeting van twee dingen: *wat de opgave wil tonen* + *welk attribuut je component verwacht*.

```
el.setAttribute('title', `${entry.product.name} (${entry.product.price.toFixed(2)} EUR)`)
//                ^attribuutnaam (zelfgekozen)                ^de inhoud (template literal uit je model-velden)
```

- **'title' is een zelfgekozen attribuutnaam** (geen vast woord). De enige regel: hij moet op **twee plekken exact gelijk** zijn — `setAttribute('title', ...)` in de pagina én `'title'` in `observedAttributes` + een `case 'title':` in het component. Je "haalt" `'title'` dus uit je eigen component: het is de naam die je daar koos om die tekstregel te tonen. Had je hem `'label'` genoemd, dan schreef je hier `setAttribute('label', ...)`.
- **De template literal** (tussen backticks, met `${...}`) bouwt de tekst die de **opgave** vraagt (bv. "naam + prijs op één regel"), uit je **model-velden** (`entry.product.name`, `entry.movie.title`, ...). `.toFixed(2)` = 2 decimalen, `EUR` en de haakjes zijn gewoon letterlijke tekst voor de opmaak. Resultaat: bv. `Laptop (999.00 EUR)`.
- **Eén attribuut of meerdere?** Heeft je component **één tekstplek** (één span)? → combineer alles in één attribuut (`'title'`). Heeft het **meerdere plekken** (`#name`, `#price`, ...)? → geef **losse** attributen mee (`setAttribute('name', ...)`, `setAttribute('price', ...)`), zoals op je eerste pagina. Dáárom ziet het er op de tweede pagina vaak anders uit dan op de eerste.

Vergeet niet: dit is een nieuwe pagina-klasse, dus je moet ze ook **importeren en in de router zetten** in `main.ts` (stap 1b) — net als je eerste pagina.

Een TOTAAL tonen (totale prijs / aantal / ...) — ENKEL op de pagina

Wanneer? De opgave vraagt onderaan een **totaal**: totale prijs, aantal items, som van porties, ... (bv. "Toon ook de *totaalprijs*").

Waar? Alleen op de pagina (in `render()`), **niet** in het component. Een totaal gaat over de **hele collectie**; een los item-component kent enkel zichzelf en weet niets van de andere items. Dus: één label in de pagina-HTML + de berekening in de pagina.

Hoe (2 dingen):

(1) Verwijzing naar het totaal-label (bovenaan de klasse; id uit je HTML — bv. `<span id="cart-total">`):

```
#total = this.body.querySelector<HTMLSpanElement>('#cart-total')!
```

(2) In `render()`, ná de `.map(...)`, optellen met `reduce` en in het label zetten:

```
// som van een veld over de hele collectie. Het 2e argument (0) is de startwaarde.
const total = this.#collection.reduce((sum, entry) => sum + entry.product.price, 0)
this.#total.innerText = total.toFixed(2) // toon met 2 decimalen
```

De kern:

- `reduce((sum, entry) => sum + entry.<veld>, 0)` telt een veld op over alle items. Begin altijd met `0`.
 - totale **prijs** → `sum + entry.product.price`
 - **aantal** items → gewoon `this.#collection.length`
 - som van **porties/aantallen** → `sum + entry.amount`
- Zet het resultaat in het label met `.innerText`. Bedrag? `.toFixed(2)`.
- Omdat dit in `render()` staat, klopt het totaal **automatisch** telkens de collectie wijzigt (de observer hertekent).

⚠ **Kijk wat al in de HTML staat.** Vaak staat de eenheid er al: `Totale prijs: <span id="cart-total">0.00</span> EUR`. Dan zet je enkel het **getal** in het span (`total.toFixed(2)`) — schrijf je `total.toFixed(2) + ' EUR'`, dan staat "EUR" er dubbel.

Het item-component voor de tweede pagina afwerken (anders zie je placeholder-tekst!)

Gebruikt de tweede pagina een **apart** component (bv. `cartItem` / `watchlistItem`)? Dan is dat in de startbestanden meestal nog een **lege stub** zonder `attributeChangedCallback`. Werk het af zoals je item-kaart in stap 2b, maar dan minimaal — enkel de attributen die je in stap 6 met `setAttribute(...)` doorgeeft (vaak `title`):

```
import {CustomElement} from '../router/customElement.ts'
import HTML from './secondItem.html?raw'

export class SecondItem extends CustomElement {
  // 🐞 elk attribuut dat de pagina met setAttribute zet, moet hier in observedAttributes staan
  static observedAttributes = ['title']

  // 🐞 verwijzing naar de plek in de HTML waar de tekst moet komen (id uit secondItem.html)
  #label = this.componentBody.querySelector<HTMLSpanElement>('#label')!

  constructor() {
    super(HTML)
  }

  attributeChangedCallback(name: string, _oldValue: string, newValue: string) {
    switch (name) {
      case 'title':
        this.#label.innerText = newValue // 🐞 toont bv. "Inception (2010)"
        break
    }
  }
}
```

Symptoom als je dit vergeet: je items verschijnen wél (juiste aantal), maar tonen allemaal de **standaardtekst uit de HTML** (bv. "Een film") i.p.v. de echte titel. Oorzaak: het `title`-attribuut komt binnen, maar niemand vangt het op. (Het verwijderen via de delete-knop is stap 7 — dat voeg je later toe.)

Knop/icoon verbergen op één pagina (bv. vuilbak verbergen op de tweede pagina, zoals 2025-06): geef een attribuut `hide-delete="true"` mee en verberg in het component: `this.#deleteBtn.hidden = newValue === 'true'`.

✓ **Test:** de tweede pagina toont de opgeslagen collectie (+ eventueel totaalprijs).

STAP 7 — Updaten / verwijderen uit de collectie (2-3 punten)

Doel: een item uit de collectie halen — **enkel in localStorage**, niet uit de database.

Waar: in het **collectie-item-component** van de tweede pagina (`components/cartItem/...` of `watchListItem/...`) — zie het ⚠️-kader bij Variant B over wélk component.

Variant A — verwijderen via custom event + `update` (bv. vraag uit quiz, 2025-06)

In `second.ts`, per item-element:

```
el.addEventListener('removeFromCollection', () => {
  void collectionLocalProvider.update(activeCollection.id, {
    ... activeCollection,
    items: activeCollection.items.filter(i => i.id !== entry.id), // 🗑️ item eruit filteren
  })
})
```

Variant B — verwijderen RECHTSTREEKS via de provider, GEEN event (bv. uit winkelmandje, 2025-08)

Sommige opgaven eisen voor de max-score **geen** custom event hier. Dan spreekt het component **zelf** de provider aan.

⚠️ **In WELK component?** Er zijn vaak twee verschillende delete-knoppen, en het is makkelijk te verwarren:

- De vuilbak op de **eerste pagina (item-kaart)** = item **uit de database** verwijderen → `restProvider.delete(...)` (stap 4).
- De X op de **tweede pagina (collectie-item)** = item **uit localStorage** verwijderen → `localProvider.delete(...)` (deze stap 7).

Zet stap 7 dus in het **tweede-pagina-component** (`cartItem.ts` / `watchListItem.ts`), NIET in de item-kaart van de eerste pagina. Reden: `this.id` moet het id van het **collectie-item** zijn (door de tweede pagina gezet met `setAttribute('id', entry.id)`). In de item-kaart is `this.id` het id van het **bron-item** (bv. de film/het product), en daarmee vindt `localProvider.delete(...)` niets om te verwijderen.

Dit zit in het **component** (bv. `cartItem.ts` / `watchListItem.ts`). Je hebt drie dingen nodig: **(1)** de provider importeren, **(2)** de delete-knop opzoeken als veld, **(3)** in de constructor een click-listener die `delete(this.id)` aanroept. Hier het component **in zijn geheel** (let op de drie 🗑️-regels):


```
import {CustomElement} from '../router/customElement.ts'
import HTML from './cartItem.html?raw'
import {collectionLocalProvider} from '../data/data.ts' // 🐞 (1) jouw provider-naam uit data.ts

export class SecondItem extends CustomElement {
  static observedAttributes = ['title', 'id']

  #label = this.componentBody.querySelector<HTMLSpanElement>('#label')!
  // 🐞 (2) de delete-knop opzoeken – gebruik het echte id uit JOUW html (bv. '#delete-btn')
  #deleteBtn = this.componentBody.querySelector<HTMLButtonElement>('#delete-btn')!

  constructor() {
    super(HTML)

    // 🐞 (3) bij klik rechtstreeks de provider aanspreken (geen custom event)
    this.#deleteBtn.addEventListener('click', () => {
      void collectionLocalProvider.delete(this.id) // this.id = id van het collectie-item (door de pagina gez
    })
  }

  attributeChangedCallback(name: string, _oldValue: string, newValue: string) {
    switch (name) {
      case 'title':
        this.#label.innerText = newValue
        break
    }
  }
}
```

Werkt de knop niet (er gebeurt niets)? Lopen deze drie punten na:

1. Heb je het `#deleteBtn` -veld wel aangemaakt met het juiste id (`querySelector('#delete-btn')`)? Zonder dat veld bestaat `this.#deleteBtn` niet.
2. Staat de `addEventListener` in de **constructor** (niet ergens buiten de klasse)?
3. Importeer je de **juiste provider-naam** uit `data.ts` ? `collectionLocalProvider` is hier een plaatshouder — bij jou heet die bv. `cartLocalProvider` of `moviesWatchListProvider` .

Onthoud het verschil (de opgave vraagt het bewust verschillend):

- iets dat de **parent/pagina** moet beslissen → **custom event** (`dispatchEvent`).
- iets dat het component **zelf** mag doen (rechtstreeks data aanpassen) → **provider rechtstreeks aanroepen**.

✅ **Test:** item verdwijnt uit de collectie maar blijft in de database (op de eerste pagina).

Snelle checklist tijdens het examen

- ☐ Voor elke **pagina** en elk **custom element** een minimaal `.ts` -bestand gemaakt dat `super(HTML)` doet (anders zie je niets in vraag 1)
- ☐ Custom elements **geregistreerd** in `main.ts` + routes gezet
- ☐ Navbar bovenaan elke pagina + links werken
- ☐ Provider(s) aangemaakt in `data.ts` (REST = API, Local = localStorage met juiste key)
- ☐ Pagina: **observer** + `getAll()` + `render()` (de heilige drievuldigheid)
- ☐ Attributen altijd **strings** en **kebab-case**; **id** **meeggeven**; arrays via `JSON.stringify` / `JSON.parse`
- ☐ Filter in een **aparte functie**, alles met `&&` zodat filters samenwerken
- ☐ Verwijderen uit DB → **REST**-provider `.delete()`
- ☐ Toevoegen/selecteren → **custom event** + opslaan via **Local**-provider
- ☐ Knopstaat (+ / - / ✓) via een attribuut als `is-added`
- ☐ Verwijderen uit collectie: event + `update`, **of** rechtstreeks `.delete()` (lees de opgave!)
- ☐ Alles **strongly typed** (TypeScript) — geen `any`
- ☐ Na ELKE stap testen in de browser

Veelvoorkomende valkuilen

- **Element niet geregistreerd** → tag verschijnt leeg. Eerst `customElements.define( ... )`.
 - **camelCase attribuut** → komt niet binnen in `attributeChangedCallback`. Gebruik kebab-case.
 - **Attribuut staat niet in `observedAttributes`** → `attributeChangedCallback` loopt niet voor dat attribuut.
 - **`id` vergeten mee te geven** → verwijderen werkt niet (`this.id` is leeg).
 - **Form herlaadt de pagina** bij de filterknop → `evt.preventDefault()`.
 - **Rommeldata in localStorage** → wis de localStorage van localhost in DevTools en herstel de json-backup van de server.
 - **Vergeten te herrenderen** na een wijziging → roep `this.render()` op (of laat het via de observer gebeuren).
-

EXTRA HOOFDSTUK — Minder voorkomende vragen

Dit hoofdstuk staat **los** van het stappenplan hierboven. Het basisexamen heb je met stappen 1-7. Hieronder staan extra mechanismen die **soms** gevraagd worden (zoals in het receptenexamen). Pak ze er enkel bij als de opgave er expliciet om vraagt — anders negeer je dit volledig.

Elk extra hoofdstuk volgt **dezelfde structuur als de stappen hierboven** (Doel/Wanneer → Waar → Hoe → De kern → ⚠️ Let op → ✅ Test) en is **zelfstandig leesbaar**. Veel van deze extra's bouwen op het **custom-event-principe uit stap 5b/5c**: de kaart vuurt een event af, de pagina luistert — en die event-naam moet op **beide** plekken exact gelijk zijn.

EXTRA A — Een nieuw item AANMAKEN in de database via een formulier (POST)

Wanneer? De opgave geeft een **formulier** (inputs + een submit-knop) waarmee de gebruiker een **nieuw record** toevoegt dat in de **database** terechtkomt. Kenmerk: "voeg een nieuw X toe via de **RestPersistenceProvider**". (Sla je in localStorage op? Dan is het gewoon **create** zoals stap 5, geen formulier-specifieke aanpak nodig.)

Waar? Op de pagina met dat formulier (meestal de **eerste pagina**). Je hebt het formulier al als HTML; jij voegt enkel de logica toe.

Hoe (3 dingen):

(1) Verwijzingen naar het formulier en zijn velden (bovenaan de klasse; gebruik de echte id's uit je HTML):

```
#addForm = this.body.querySelector<HTMLFormElement>('#add-form')!
#nameInput = this.body.querySelector<HTMLInputElement>('#new-name')!
#priceInput = this.body.querySelector<HTMLInputElement>('#new-price')!
#categorySelect = this.body.querySelector<HTMLSelectElement>('#new-category')!
```

(2) In de constructor: luister op **submit, bouw een object, roep **create** aan:**

```
this.#addForm.addEventListener('submit', async evt => {
  evt.preventDefault() // ⚠ verplicht: anders herlaadt het formulier de pagina en ben je alles kwijt

  // ✎ Omit<Item, 'id'> = "een Item, maar zonder id". Het id wordt door de SERVER toegekend,
  // dus dat stuur je NIET mee. Alle andere velden moet je wél invullen.
  const newItem: Omit<Item, 'id'> = {
    name: this.#nameInput.value, // tekst → rechtstreeks
    price: Number(this.#priceInput.value), // getal → input.value is een STRING, dus omzetten
    category: this.#categorySelect.value as Item['category'], // enum-achtig veld → cast houdt het strongly
  }

  await itemRestProvider.create(newItem) // POST → server geeft het nieuwe item terug + verwittigt observer
  this.#addForm.reset() // (3) formulier leegmaken zodat de gebruiker opnieuw kan invullen
})
```

De kern (algemeen toepasbaar):

1. `evt.preventDefault()` op de form- **submit** (anders full-page reload).
2. Bouw een object van type `Omit<Item, 'id'>` uit de inputwaarden. `input.value` is **altijd een string**; gebruik `Number(...)` voor getallen, en `as Item['veld']` voor een vast keuzeveld (bv. `'easy' | 'medium' | 'hard'`).
3. `await provider.create(newItem)` → de observer (uit stap 2) hertekent → het nieuwe item verschijnt vanzelf.
4. `form.reset()` maakt alle velden in één keer leeg.

Waarom `Omit<Item, 'id'>`? Je `create` -methode verwacht een item zónder id (de server maakt het). Zou je `id` wél meegeven, dan klaagt TypeScript. Vul je een veld niet in, dan klaagt TypeScript óók ("property ... ontbreekt") — handig, want zo vergeet je geen veld.

Validatie nodig? Zet `required` op de inputs in de HTML (dan blokkeert de browser een lege submit), of check zelf vóór `create` (`if (!this.#nameInput.value) return`).

✅ **Test:** vul het formulier in en verzend → het nieuwe item verschijnt in de lijst en blijft staan na refresh (het zit in de database).

EXTRA B — Een item UPDATEN in de database (PUT) · (bv. "korting toepassen", score aanpassen, status wijzigen)

Wanneer? De opgave vraagt om een **waarde van een bestaand record aan te passen** en **permanent** op te slaan in de database. Voorbeelden: een **korting** op de prijs, een score verhogen, een status omzetten. Kenmerk: er staat "sla op via de **RestPersistenceProvider**" en de wijziging moet behouden blijven (ook na refresh).

Waar: in het **item-component** (de knop) + op de **pagina** (de listener die **update** aanroept).

Dit is **dezelfde structuur als toevoegen-aan-collectie (stap 5)**, maar in plaats van **create** / **delete** roep je **update(...)** aan. Twee helften:

Helft 1 — in de kaart (component): de knop vuurt een custom event af (zoals stap 5b). Zoek **eerst** de knop op als veld (bovenaan de klasse, met het id uit je HTML), en hang er dan de listener aan in de constructor:

```
// bovenaan de klasse: verwijzing naar de knop (id uit je component-HTML)
#discountBtn = this.componentBody.querySelector<HTMLButtonElement>('#discount-button')!

// in de constructor:
this.#discountBtn.addEventListener('click', () => {
  this.dispatchEvent(new CustomEvent('applyDiscount')) // 🐞 naam kies je zelf
})
```

Net als bij elke knop in een component: je kan er pas een listener op hangen nadat je hem hebt opgezocht met **querySelector**. Vergeet dat veld niet, anders bestaat **this.#discountBtn** niet.

Helft 2 — in de pagina: luister, bereken de nieuwe waarde, en sla op met **update :**

```
// in render(), binnen je .map(item => { ... }), op het element dat je aanmaakt:
el.addEventListener('applyDiscount', async () => { // 🐞 zelfde naam als in de kaart
  const updated = { ...item, price: item.price * 0.9 } // 🐞 enkel het veld dat wijzigt herberekenen
  await itemRestProvider.update(item.id, updated) // PUT → observer → UI ververs vanzelf
})
```

De kern (algemeen toepasbaar):

1. Bereken de **nieuwe waarde** uit de huidige (**item.price * 0.9**, **item.rating + 0.5**, **!item.done**, ...).
2. Bouw een nieuw object met **spread + het gewijzigde veld**: **{ ... item, veld: nieuweWaarde }**. Zo behoud je alle andere velden en overschrijf je er één.
3. Roep **provider.update(item.id, updated)** aan op de juiste provider:
 - **RestPersistenceProvider** → wijziging gaat naar de **database** (permanent, blijft na refresh). ← korting hoort hier
 - **LocalStoragePersistenceProvider** → wijziging in **localStorage** (bv. aantal/porties, zie EXTRA E).
4. Je hoeft **niet** zelf te herrenderen: **update()** verwittigt de observers → de UI ververs automatisch.

Korting permanent? Ja — omdat de REST-provider naar het json-bestand op de server schrijft, blijft de nieuwe prijs staan. Resetten doe je via het backup-bestand (zie de TIP in de opgave).

Heb je bij de knop ook moeten weten **hoeveel** of **welke richting** (bv. +/- knoppen)? Geef dat mee via **CustomEvent-detail** — zie **EXTRA C**.

⚠ **Event-naam:** net als in stap 5b/5c moet de naam in `dispatchEvent(new CustomEvent('applyDiscount'))` (kaart) exact gelijk zijn aan die in `addEventListener('applyDiscount', ...)` (pagina). Anders gebeurt er niets bij klikken.

✅ **Test:** klik de knop → de waarde verandert (bv. prijs * 0.9) en blijft behouden na refresh (het staat in de database).

EXTRA C — Een CustomEvent met `detail` (extra data meegeven)

Wanneer? Een gewoon custom event (stap 5b) zegt enkel "er is geklikt". Soms moet de kaart óók **meegeven hoeveel of welke richting** — bv. twee knoppen `+` en `-` die allebei "verander de waarde" betekenen, maar de ene met `+1` en de andere met `-1`. Daarvoor hangt een `CustomEvent` een **`detail`-object** mee: `new CustomEvent('x', { detail: { ... } })`. Zonder die behoefte heb je `detail` niet nodig (dan blijft het gewoon stap 5b).

Waar: in het **component** (de `+` / `-` knoppen vuren af) + op de **pagina** (de listener leest `detail` uit). Zelfde 2-helften-principe als stap 5b/5c.

Verschil met een gewoon custom event:

	gewoon (stap 5b)	met <code>detail</code> (EXTRA C)
afvuren	<code>new CustomEvent('addToCart')</code>	<code>new CustomEvent('changeAmount', { detail: { delta: 1 } })</code>
betekenis	"knop geklikt"	"knop geklikt + deze extra info "

Helft 1 — in het component: detail meegeven. Eén event-naam, twee knoppen met een andere `delta`:

```
this.#upBtn.addEventListener('click', () => {
  this.dispatchEvent(new CustomEvent('changeAmount', { detail: { delta: 1 } })))
})
this.#downBtn.addEventListener('click', () => {
  this.dispatchEvent(new CustomEvent('changeAmount', { detail: { delta: -1 } })))
})
```

Helft 2 — in de pagina: detail uitlezen. Let op: een listener krijgt het algemene type `Event`, dat geen `detail` kent. Daarom **cast** je naar `CustomEvent<...>` om strongly typed bij `detail` te komen:

```
el.addEventListener('changeAmount', async evt => {
  const delta = (evt as CustomEvent<{ delta: number }>).detail.delta // ← de cast haalt 'detail' tevoorschijn
  const newValue = Math.max(1, item.amount + delta) // huidige waarde + delta (hier min. 1)
  await provider.update(item.id, { ...item, amount: newValue }) // opslaan (REST of Local, zie B/E)
})
```

De kern:

1. In de kaart: `dispatchEvent(new CustomEvent('naam', { detail: { ...wat je nodig hebt } })))`.
2. In de pagina: `const x = (evt as CustomEvent<{ ... }>).detail.x`.
3. Vaak combineer je dit met **EXTRA B** (opslaan in DB) of **EXTRA E** (opslaan in localStorage).

Waarom de cast (`evt as CustomEvent<{ ... }>`)? De parameter `evt` is van type `Event` (zo typeert `addEventListener` het), en `Event` heeft geen `detail`. Met de cast vertel je TypeScript: "dit is een `CustomEvent` met een detail van deze vorm", zodat `.detail.delta` herkend en strongly typed is.

Alternatief zonder detail: twee aparte event-namen (`'increase'` / `'decrease'`) en twee listeners. Werkt ook, maar met `detail` heb je één event-naam en minder herhaling.

✅ **Test:** klik `+` en `-` → de waarde gaat met de juiste stap omhoog/omlaag (en wordt opgeslagen via je `update`).

EXTRA D — SORTEREN (bovenop filteren)

Wanneer? De opgave vraagt de lijst te **sorteren** via een dropdown (bv. op prijs, score, naam — oplopend/aflopend). Meestal bovenop de filter (stap 3): eerst filteren, dan sorteren.

Waar? Op dezelfde pagina als je filter. Je hebt een extra `<select>` voor de sorteeroptie nodig.

Hoe (3 dingen):

(1) Verwijzing naar de sorteer-dropdown (id uit je HTML):

```
#sortSelect = this.body.querySelector<HTMLSelectElement>('#sort-select')!
```

(2) Hertekenen wanneer de keuze verandert — net als bij de filter, ofwel op de filterknop, ofwel op `change`:

```
this.#sortSelect.addEventListener('change', () => this.render())
```

(3) In een hulpfunctie: eerst filteren, dan sorteren. Sorteer altijd een **kopie** met `[... arr]`:

```
#filterAndSort(): Item[] {
  const filtered = this.#items.filter(i => this.#matchesFilter(i)) // eerst filteren (stap 3)

  switch (this.#sortSelect.value) {
    case 'price-desc': return [...filtered].sort((a, b) => b.price - a.price) // dan sorteren op de gekozen optie // getal: hoog → laag
    case 'price-asc':  return [...filtered].sort((a, b) => a.price - b.price) // getal: laag → hoog
    case 'name-asc':   return [...filtered].sort((a, b) => a.name.localeCompare(b.name)) // tekst: A → Z
    default:           return filtered // geen sortering gekozen
  }
}
```

En in `render()` gebruik je die functie i.p.v. `this.#items.filter(...)`:

```
this.#filterAndSort().map(item => { /* ... element maken, attributen zetten, appendChild ... */ })
```

De kern (de twee dingen die je moet onthouden):

- **Getallen** sorteer je met een aftrekking: `(a, b) => a.price - b.price` is **oplopend** (laag→hoog), `(a, b) => b.price - a.price` is **aflopend** (hoog→laag).
- **Tekst** sorteer je met `(a, b) => a.name.localeCompare(b.name)` (A→Z); omdraaien = `b.name.localeCompare(a.name)`.

⚠ **Sorteer altijd een KOPIE:** `[... filtered].sort(...)`. `sort()` wijzigt de array **zelf** (in-place) en geeft hem ook terug. Doe je `this.#items.sort(...)`, dan herorden je je originele lijst permanent — wat rare effecten geeft bij een volgende render of filter. Met `[... filtered]` maak je een nieuwe array en blijft het origineel intact.

✓ **Test:** kies elke sorteeroptie → de lijst staat in de juiste volgorde, en de filter blijft tegelijk werken.

EXTRA E — Een VELD van een localStorage-item updaten (bv. aantal/porties)

Wanneer? Een collectie-item in localStorage heeft een **teller** (aantal, porties, hoeveelheid) die de gebruiker met **+** / **-** moet kunnen aanpassen. Je **verwijderd** het item dus niet — je **wijzigt één veld** ervan. Kenmerk: "verhoog/verlaag het aantal" en het moet bewaard blijven in **localStorage**.

Voorwaarde: je collectie-model heeft zo'n veld, bv. `interface CartItem { id, product, amount }.`

Waar? Op de pagina die de collectie toont (de tweede pagina, stap 6), en de **+** / **-** knoppen zitten in het collectie-item-component.

Hoe — dit is EXTRA C (event met `detail`) + `update` op de Local-provider:

Helpt 1 — in het component (`CartItem` / `menuItem`): **+ / **-** sturen het verschil mee:**

```
this.#upBtn.addEventListener('click',
  () => this.dispatchEvent(new CustomEvent('changeAmount', { detail: { delta: 1 } })))
this.#downBtn.addEventListener('click',
  () => this.dispatchEvent(new CustomEvent('changeAmount', { detail: { delta: -1 } })))
```

Helpt 2 — in de pagina: nieuw aantal berekenen en opslaan in localStorage:

```
el.addEventListener('changeAmount', async evt => {
  const delta = (evt as CustomEvent<{ delta: number }>).detail.delta
  const newAmount = Math.max(1, entry.amount + delta) // 🚫 niet onder 1 zakken
  await collectionLocalProvider.update(entry.id, { ...entry, amount: newAmount }) // update in localStorage
})
```

Totaal tonen (komt vaak samen voor) — met `reduce` :

```
#totalLabel = this.body.querySelector<HTMLSpanElement>('#total')! // veld bovenaan, id uit je HTML
// ... in render(), na de map:
const total = this.#collection.reduce((sum, e) => sum + e.amount, 0) // som van alle aantallen
this.#totalLabel.innerText = total.toString()
```

De kern:

1. **+** / **-** → custom event met `detail.delta` (EXTRA C).
2. Nieuw aantal = `Math.max(1, entry.amount + delta)` (begrens zodat het niet onder 1 gaat).
3. `collectionLocalProvider.update(entry.id, { ...entry, amount: newAmount })` → schrijft naar localStorage, verwittigt de observer → UI ververscht vanzelf.
4. Een **totaal** bereken je met `reduce((sum, e) => sum + e.amount, 0)`.

Verschil met EXTRA B: exact dezelfde `update(id, { ...item, veld })`-techniek, maar op de **Local**-provider (localStorage) i.p.v. de **REST**-provider (database). Kies de provider die de opgave noemt. **Verschil met verwijderen (stap 7):** daar haal je het item wég (`delete`); hier blijft het staan en pas je enkel een veld aan (`update`).

✅ **Test:** klik **+** / **-** op een collectie-item → het aantal wijzigt, het totaal klopt, en het blijft behouden na refresh.

Mini-checklist voor deze extra's

- ☐ Formulier → `evt.preventDefault()` + `create( ... )` (REST) + `form.reset()`
- ☐ Waarde aanpassen in DB → `update(id, { ... item, veld})` (REST)
- ☐ `+` / `-` met richting → `new CustomEvent('x', {detail: {delta}})` + cast bij uitlezen
- ☐ Sorteren → kopie `[ ... arr].sort( ... )`, ná het filteren
- ☐ Teller in localStorage → `update(id, { ... item, amount})` (Local) + `reduce` voor het totaal
- ☐ Input-waarden zijn strings → `Number( ... )` voor getallen; enums via `... as Item['veld']`